# CALPHAD Server & KKS Framework Guide

*Reference for the APIs, the KKS module and the clients*

**Prepared for**
Gaurav Rathod

**Companion study guide**
*CALPHAD Study Guide: From Gibbs Energy to the KKS Server*

*6 November 2026*

# Contents

**1. Overview** **6**

Components . . . . . 6

Server or in-process? . . . . . 7

What the framework adds to pycalphad . . . . . 7

How the parts connect . . . . . 7

**2. Installation and quick start** **7**

Requirements . . . . . 7

Install and start . . . . . 8

First call, four ways . . . . . 9

**3. Conventions** **10**

Units . . . . . 10

Specifying an alloy . . . . . 10

Energy reference . . . . . 11

Phase names and order/disorder phases . . . . . 11

Missing values . . . . . 11

Errors . . . . . 11

**4. Server and database management** **12**

Starting the server . . . . . 12

Endpoints for databases . . . . . 13

Example: list what is loaded . . . . . 13

Example: load another database . . . . . 13

Example: inspect a database . . . . . 13

The same without a server . . . . . 14

Behaviour worth knowing . . . . . 14

**5. API 1: Equilibrium — `POST /equilibrium` 14**

Request . . . 15

Response . . . 15

The stability check . . . 16

Example 5.1 — one temperature . . . 16

Example 5.2 — temperature sweep: solidus and liquidus . . . 17

Example 5.3 — weight percent . . . 17

Example 5.4 — restricting phases: metastable equilibrium . . . 17

Example 5.5 — ternary with ordered phases . . . 18

Example 5.6 — one phase twice: $\gamma + \gamma'$ and a miscibility gap . . . 18

**6. API 2: Phase energies — `POST /phase_energies` 19**

What it computes . . . 19

Request . . . 19

Response . . . 19

Example 6.1 — a two-phase alloy . . . 20

Example 6.2 — an ordered phase and compounds . . . 21

Example 6.3 — a phase that would rather split . . . 21

Example 6.4 — ranking phases by driving force . . . 21

**7. API 3: Energy surfaces — `POST /energy_surface` 22**

Request . . . 22

The two modes . . . 22

Response . . . 23

Example 7.1 — binary free-energy curves . . . 23

Example 7.2 — ternary surface on a Gibbs triangle . . . 24

Example 7.3 — a binary subsystem of a larger database . . . 24

Example 7.4 — the lowest single phase at each composition . . . 24

Example 7.5 — every sampled configuration . . . . . 24
Speed . . . . . 25

**8. KKS module — `calphad_kks.py` 25**

8.1 The model in brief . . . . . 25
8.2 Constructing a model . . . . . 25
8.3 Free-energy modes . . . . . 27
8.4 Methods . . . . . 27
Example 8.1 — $\gamma$/liquid tie lines in Ni–Al . . . . . 28
Example 8.2 — split and driving force . . . . . 29
Example 8.3 — parabolic energies for speed . . . . . 29
Example 8.4 — a temperature field (additive manufacturing) . . . . . 30
Example 8.5 — table mode . . . . . 30
Example 8.6 — $\gamma'$/liquid with an ordered phase . . . . . 30
Example 8.7 — a complete solver loop (1-D, explicit) . . . . . 31
Example 8.8 — exporting for another code . . . . . 31
What the module does not do . . . . . 31

**9. KKS over HTTP and in C++ 32**

9.1 `POST /kks/parameters` . . . . . 32
9.2 `cpp/kks_parabolic.hpp` . . . . . 33
Example 9.1 — from a CSV exported in Python . . . . . 33
Example 9.2 — coefficients straight from the server . . . . . 34
Example 9.3 — inside a C++ time loop . . . . . 34

**10. Clients and other languages 35**

10.1 Python client — `calphad_client.py` . . . . . 35
10.2 C++ client — `cpp/calphad_client.hpp` . . . . . 36
10.3 Other languages . . . . . 37

**11. Validation, self-test and the 1-D demo** **37**

11.1 `selftest.py` . . . . . 37

11.2 KKS checks . . . . . 38

11.3 The 1-D Ni–10Al demo . . . . . 38

**12. Troubleshooting, performance and limitations** **39**

Troubleshooting . . . . . 40

Performance . . . . . 41

Limitations . . . . . 41

pycalphad pitfalls the framework works around . . . . . 42

# 1. Overview

The framework loads any CALPHAD `.tdb` database once and answers thermodynamic questions about it from Python, C++ or any language that speaks HTTP. It also supplies fast Kim–Kim–Suzuki (KKS) building blocks for binary, two-phase phase-field models. pycalphad does the Gibbs-energy minimisation underneath.

It answers four kinds of question:

1. **Equilibrium** (API 1): which phases are stable for a given alloy and temperature, how much of each, and their compositions.
2. **Phase energies** (API 2): the Gibbs energy of every phase at the alloy composition, stable or not, with its chemical potentials and driving force.
3. **Energy surfaces** (API 3): G(x) of every phase across the whole composition space at one temperature.
4. **KKS inputs**: tie lines, free-energy curvatures, the KKS composition split and the driving force for a phase-field solver.

## Components

| File | Role | Typical use |
|---|---|---|
| `calphad_core.py` | `CalphadSystem` class: loads a TDB and implements APIs 1–3 | Import directly in Python scripts and notebooks |
| `calphad_kks.py` | `KKSModel` class: phase free energies, tie lines, KKS split, driving force, export | Python phase-field solvers |
| `calphad_server.py` | HTTP/JSON server (FastAPI) exposing the core and `/kks/parameters` | Any language; several codes sharing one database |
| `calphad_client.py` | Python client, standard library only | Python code talking to a running server |
| `cpp/calphad_client.hpp` | C++ client (header-only) | C++ codes talking to the server |
| `cpp/kks_parabolic.hpp` | Dependency-free C++ KKS with parabolic energies | Inside a C++ solver's time loop |
| `examples/kks_1d_directional.py` | 1-D Ni–10Al directional solidification demo | Template for your own solver |
| `selftest.py` | Thermodynamic consistency checks | Run once on every new database |
| `cuni_cost507.tdb`, `alni_dupin_2001.tdb`, `alcrni.tdb` | Example databases | Cu–Ni, Al–Ni, Al–Cr–Ni |

### Server or in-process?

- **Python code**: import `calphad_core` and `calphad_kks` directly. Results are identical to the server's, with no HTTP overhead.
- **Other languages, or several programs sharing one database**: run the server and call it over HTTP.
- **Inside a time loop**: never make an HTTP call per grid point (about 1 ms each). Fetch tie lines, coefficients or tables once at start-up, then evaluate locally with `KKSModel` in Python or `kks_parabolic.hpp` in C++.

### What the framework adds to pycalphad

pycalphad finds the equilibrium state. The framework wraps it in a stable JSON interface and adds:

- composition input in mole or weight units with a balance element;
- labelling of order/disorder phases (for example, a disordered `FCC_L12` is reported as `FCC_A1`);
- a global-minimum check that re-solves with denser sampling when a phase was missed;
- multi-start minimisation of single phases, so ordered phases such as $\gamma'$ are not reported in their metastable disordered state;
- the KKS layer.

### How the parts connect

Python codes import the modules directly; every other language goes through the server. Inside a time loop, both paths evaluate the KKS formulas locally from parameters fetched once.

## 2. Installation and quick start

Install three Python packages, start the server with a database, and make a first call. The examples below all ask the same question: Cu–33Ni (at.%) at 1500 K, which is 53% liquid and 47% FCC.

### Requirements

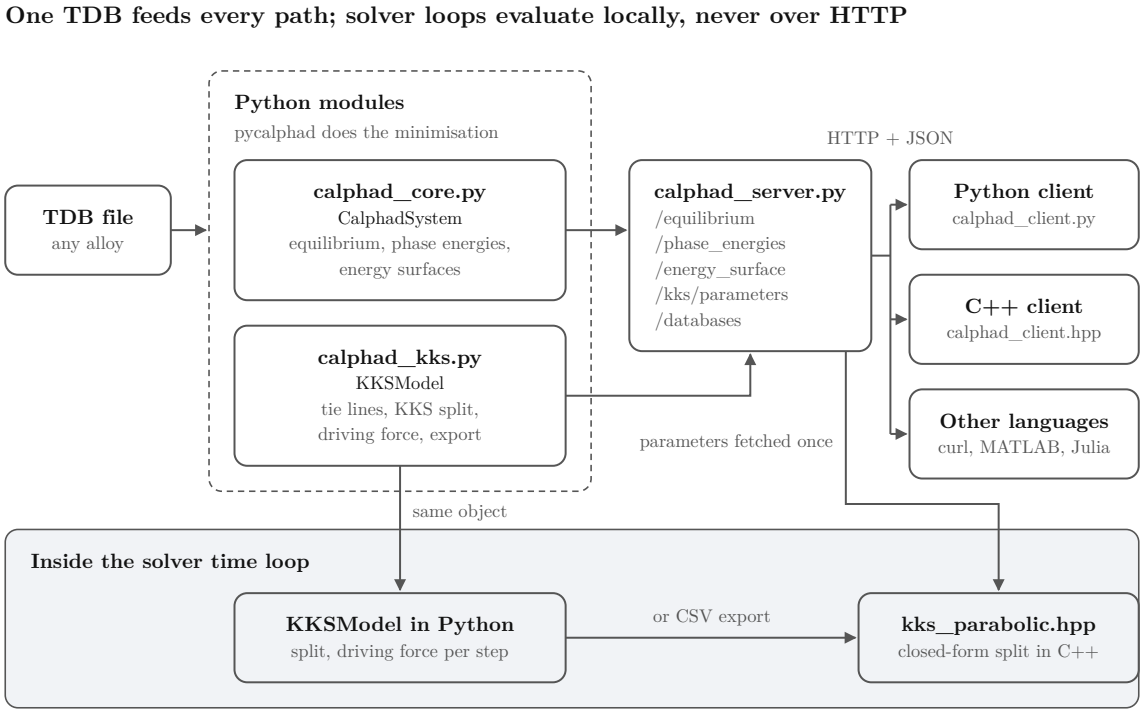



Figure 1: Framework architecture: database, Python modules, server, clients and the solver loop.

| Component | Needs | Tested with |
|---|---|---|
| Python core, KKS module, server | Python 3.10+, `pycalphad`, `fastapi`, `uvicorn` | Python 3.13, pycalphad 0.11.2 |
| Demo figures | `matplotlib` | — |
| C++ client and KKS header | C++17 compiler, CMake 3.14+ | GCC on Linux |
| C++ HTTP client only | `httplib.h` and `json.hpp` (header-only; CMake downloads them on the first build) | cpp-httplib 0.18.3, nlohmann/json 3.11.3 |

`cpp/kks_parabolic.hpp` has no dependencies at all.

## Install and start

1. Unzip `calphad_server.zip` and open a terminal in the `calphad_server` folder.
2. Install the packages:

```
pip install pycalphad fastapi uvicorn matplotlib
```

3. Optionally check the installation. The last line should read `all 28 cases passed`:

```
python selftest.py
```

4. Start the server with one or more databases:

```
python calphad_server.py --tdb cuni_cost507.tdb
```

5. Open http://127.0.0.1:8000/docs in a browser. Every endpoint can be tried there with a "Try it out" button.

## First call, four ways

**Python, no server** (fastest; use this inside Python codes):

```
from calphad_core import CalphadSystem

s = CalphadSystem.from_file("cuni_cost507.tdb")
r = s.equilibrium({"NI": 0.33}, T=1500, balance="CU")
for p in r["results"]["phases"]:
    print(p["phase"], round(p["fraction"], 4), round(p["composition"]["NI"], 4))
# LIQUID 0.5344 0.2703
# FCC_A1 0.4656 0.3985
```

**Python, through the server:**

```
from calphad_client import CalphadClient

c = CalphadClient("http://127.0.0.1:8000")
r = c.equilibrium({"NI": 0.33}, T=1500, balance="CU")
```

**curl** (or any HTTP tool):

```
curl -X POST http://127.0.0.1:8000/equilibrium \
     -H "Content-Type: application/json" \
     -d '{"composition": {"NI": 0.33}, "balance": "CU", "T": 1500}'
```

**C++:**

```
cd cpp
cmake -S . -B build && cmake --build build
./build/calphad_example 127.0.0.1 8000
```

When only one database is loaded, requests may leave out `"db"`. With several loaded, every request must name one (section 4).

# 3. Conventions

Compositions are mole fractions and energies are J per mole of atoms, relative to the database's reference state, unless a field says otherwise. T is in K and P in Pa.

## Units

| Quantity | Unit | Notes |
|---|---|---|
| Temperature `T` | K | |
| Pressure `P` | Pa | Default 101325 |
| Compositions in results | mole fraction | `composition_wt` gives mass fractions |
| Phase `fraction` | fraction of the total moles of atoms | Sums to 1 over the stable phases |
| `GM`, `GM_system`, chemical potentials | J/mol of atoms | Relative to the database reference state |
| KKS diffusion potential `mu` | J/mol | $dG/dc = \mu_{solute} - \mu_{solvent}$ |
| KKS driving force | $J/m^3$ by default | J/mol divided by `Vm`; `per_volume=False` gives J/mol |
| Molar volume `Vm` | $m^3/mol$ | You supply it (for example $7.0\times10^{-6}$ for Ni alloys) |

## Specifying an alloy

APIs 1 and 2 take the same alloy fields:

| Field | Type | Default | Meaning |
|---|---|---|---|
| `composition` | object, element → amount | required | Element names are case-insensitive |
| `units` | string | `mole_fraction` | `mole_fraction`, `atomic_percent`, `weight_fraction` or `weight_percent` |
| `balance` | string | none | Element that takes up the remainder; any value given for it is ignored |
| `P` | number | 101325 | Pressure in Pa |
| `phases` | list of strings | all | Restrict the candidate phases |

Without `balance`, the amounts must sum to 1 (or 100 for percent units) within $10^{-6}$. Weight units are converted with the database's atomic masses. Only the elements you name take part in the calculation, so a large multicomponent database can be used for any subsystem.

```
{"composition": {"NI": 0.33}, "balance": "CU"}
{"composition": {"CU": 0.67, "NI": 0.33}}
{"composition": {"NI": 30}, "units": "weight_percent", "balance": "CU"}
```

```
{"composition": {"AL": 20, "CR": 10}, "units": "atomic_percent", "balance": "NI"}
```

The first two lines are the same alloy. The third, Cu–30Ni by weight, becomes $x_{\mathrm{Ni}} = 0.31696$; the result's `alloy` field always echoes the mole fractions actually used.

## Energy reference

Energies are relative to the database's reference state, usually SER: the pure elements in their stable structure at 298.15 K and 1 bar. An absolute value such as −87 035 J/mol means nothing on its own. Differences between phases, compositions and temperatures are physical. Never mix energies from two different databases.

## Phase names and order/disorder phases

- Phase names are exactly as in the TDB, in upper case.
- Many databases describe an ordered phase and its disordered parent as one phase: `FCC_L12` covers both $\gamma'$ and $\gamma$, and `BCC_B2` covers both B2 and A2. pycalphad calculates with the ordered one only, so disordered $\gamma$ comes back named `FCC_L12`.
- Each phase result therefore carries `state` (`"ordered"`, `"disordered"` or `null` for phases that cannot order) and `equivalent_to` (the phase it really is). `{"phase": "FCC_L12", "state": "disordered", "equivalent_to": "FCC_A1"}` is $\gamma$.
- `composition_set` numbers repeats of one phase. A `2` means a second instance: a miscibility gap, or ordered and disordered states side by side, such as $\gamma + \gamma'$. API 2 lists such repeats as `"FCC_L12#2"`.
- By default every phase that can form from the named elements is a candidate. When both members of an order/disorder pair qualify, the ordered one is kept. Each result's `candidate_phases` shows the list used: asking Ni–Al for `["FCC_A1", "BCC_B2", "BCC_A2", "LIQUID"]` gives `["BCC_B2", "FCC_A1", "LIQUID"]`.
- `GET /databases/{db}` shows each phase's sublattices and any `order_disorder_pair`.

## Missing values

JSON has no NaN, so an undefined number is `null`. Example: the Gibbs energy of a stoichiometric compound such as `AL3NI1` at a composition it cannot reach.

## Errors

| HTTP status | Cause | Example `detail` |
|---|---|---|
| 400 | Bad input | `Unknown element(s) ['FE']. Database has ['CU', 'NI'].` |
| 400 | Composition does not add up | `Composition sums to 0.3, not 1. Give all elements or name a 'balance' element.` |
| 400 | Several databases loaded, no `db` given | `Several databases loaded ['alcrni', 'alni_dupin_2001', 'cuni_cost507']; say which with 'db'.` |
| 400 | Request too large | `501501 grid points for 3 elements at step=0.001; increase step or max_points (200000).` |
| 404 | Database not loaded | `Database 'cuni' not loaded. Loaded: [...]` |
| 422 | Missing or wrongly typed field | `{"type": "missing", "loc": ["body", "T"], "msg": "Field required"}` |
| 500 | pycalphad failed | `<ExceptionType>: <message>` |

In Python, bad input raises `CalphadError` (a `ValueError`). The Python client raises `CalphadServerError("HTTP 400: ...")`; the C++ client throws `std::runtime_error` with the same text.

# 4. Server and database management

The server holds any number of databases in memory, each under a name, and every calculation request says which one to use.

## Starting the server

```
python calphad_server.py --tdb FILE [--tdb FILE ...] [--host HOST] [--port PORT]
```

| Option | Default | Meaning |
|---|---|---|
| `--tdb` | none | TDB file to load; repeat for several. Registered under its file name without `.tdb` |
| `--host` | `127.0.0.1` | Only this machine can connect. `0.0.0.0` opens it to your network; there is no authentication, so only do that on a trusted network |
| `--port` | `8000` | TCP port |

```
python calphad_server.py --tdb cuni_cost507.tdb --tdb alni_dupin_2001.tdb --tdb alcrni.tdb
```

Leave it running in its own terminal. Databases added later over HTTP last until the server stops.

## Endpoints for databases

| Method and path | Body | Returns |
|---|---|---|
| `GET /health` | — | `{"status": "ok", "databases": [...]}` |
| `GET /databases` | — | Each loaded database with its elements and phases |
| `POST /databases` | `path` or `tdb_text`, optional `name` | The new database's full description |
| `GET /databases/{db}` | — | Elements, atomic masses, phases, sublattices, order/disorder pairs |
| `GET /docs` | — | Interactive documentation in the browser |
| `GET /openapi.json` | — | Machine-readable description of every endpoint |

## Example: list what is loaded

```
curl http://127.0.0.1:8000/databases
```

```
{"cuni_cost507": {"elements": ["CU", "NI"], "phases": ["FCC_A1", "LIQUID"]},
 "alcrni": {"elements": ["AL", "CR", "NI"], "phases": ["B2", "BCC_A2", "FCC_A1", "L12_FCC",
    "LIQUID"]},
 "alni_dupin_2001": {"elements": ["AL", "NI"],
                     "phases": ["AL3NI1", "AL3NI2", "AL3NI5", "BCC_A2", "BCC_B2", "FCC_A1",
    "FCC_L12", "LIQUID"]}}
```

## Example: load another database

By a path on the server's machine:

```
curl -X POST http://127.0.0.1:8000/databases -H "Content-Type: application/json" \
     -d '{"path": "/data/tdb/my_superalloy.tdb", "name": "ni_sx"}'
```

By sending the file's contents from your own machine (Python client):

```
c.upload_tdb("my_superalloy.tdb", name="ni_sx")
```

Loading under a name that already exists replaces that database.

## Example: inspect a database

```
curl http://127.0.0.1:8000/databases/cuni_cost507
```

```
{"name": "cuni_cost507",
 "elements": ["CU", "NI"],
 "atomic_masses": {"CU": 63.546, "NI": 58.69},
```

```
 "phases": {
   "FCC_A1": {"site_ratios": [1.0, 1.0], "constituents": [["CU", "NI"], ["VA"]]},
   "LIQUID": {"site_ratios": [1.0], "constituents": [["CU", "NI"]]}}}
```

For `alni_dupin_2001`, the `FCC_L12` entry also shows its order/disorder pair:

```
"FCC_L12": {"site_ratios": [0.75, 0.25, 1.0],
            "constituents": [["AL", "NI"], ["AL", "NI"], ["VA"]],
            "order_disorder_pair": {"ordered": "FCC_L12", "disordered": "FCC_A1"}}
```

### The same without a server

```
from calphad_core import CalphadSystem

s = CalphadSystem.from_file("alni_dupin_2001.tdb")     # or CalphadSystem.from_text(tdb_string,
    name="x")
s.name, s.elements, s.phases                             # 'alni_dupin_2001', ['AL', 'NI'], [...]
s.info()                                                 # same dict as GET /databases/{db}
```

### Behaviour worth knowing

- The first calculation for a new set of elements and phases builds and compiles the thermodynamic models, which takes about a second. Later calls reuse them.
- Calls on the same database are processed one at a time.
- `/kks/parameters` keeps each KKS model it builds, so repeating a request with the same settings is fast.

## 5. API 1: Equilibrium — `POST /equilibrium`

Given an alloy and a temperature, API 1 returns the stable phases, their fractions, their compositions and site fractions, and the system's chemical potentials. It is the "read the phase diagram" call.

| Access | Call |
|---|---|
| HTTP | `POST /equilibrium` |
| Python, in-process | `CalphadSystem.equilibrium(composition, T, P=101325, phases=None, units="mole_fraction", balance=None, verify=True)` |
| Python client | `CalphadClient.equilibrium(composition, T, P=None, phases=None, units=None, balance=None, verify=None, db=None)` |
| C++ client | `CalphadClient::equilibrium(composition, T or vector<double>, balance, extra)` |

## Request

The alloy fields of section 3 (`composition`, `units`, `balance`, `P`, `phases`), plus:

| Field | Type | Default | Meaning |
|---|---|---|---|
| `db` | string | the only database | Which database |
| `T` | number or list of numbers | required | K. A list returns one result per temperature |
| `verify` | boolean | `true` | Run the global-minimum check described below |

## Response

| Field | Meaning |
|---|---|
| `alloy` | Mole fractions actually used |
| `units_in` | Units of the input composition |
| `candidate_phases` | Phases considered |
| `results` | One result object, or a list of them when `T` was a list |
| `results.T`, `results.P` | Conditions |
| `results.converged` | `true` when every stable phase has a valid fraction |
| `results.GM_system` | Gibbs energy of the equilibrium mixture, J/mol |
| `results.chemical_potentials` | $\mu$ of each element, J/mol: the common tangent plane |
| `results.phases[]` | Stable phases, largest fraction first |
| `results.stability_check` | Outcome of the global-minimum check |

Each entry of `phases[]`:

| Field | Meaning |
|---|---|
| `phase` | Phase name as in the database |
| `composition_set` | 1, or 2+ for a repeat of the same phase |
| `state`, `equivalent_to` | Order/disorder label (section 3) |
| `fraction` | Fraction of the total moles of atoms |
| `composition` | Mole fractions in this phase |
| `composition_wt` | Mass fractions in this phase |
| `GM` | This phase's Gibbs energy at its own composition, J/mol ($= \Sigma\ x_i \mu_i$) |
| `site_fractions` | One object per sublattice: species $\rightarrow$ site fraction |

## The stability check

pycalphad starts its minimiser from a sampled set of configurations. If the sample misses a phase's low-energy region, the answer is a local minimum. With `verify: true` (the default), every absent phase must have a driving force of at most 1 J/mol against the result's chemical potentials. If one is higher, the calculation is repeated with denser sampling: 2000, then 10 000 points per phase.

| Field | Meaning |
|---|---|
| `verified` | `true` when no absent phase could lower the energy |
| `sampling_density` | `"default"`, `2000` or `10000`: the density that passed |
| `driving_forces_absent_phases` | Driving force of each absent phase, J/mol; all $\leq 0$ when verified |

Why it matters: Ni–20Al–10Cr at 1600 K with `verify: false` returns $L1_2$ 0.747 + liquid 0.253 at −118 162.44 J/mol. The check finds B2 with a positive driving force and re-solves: $L1_2$ (disordered, so $\gamma$) 0.856 + B2 0.145 at −118 173.48 J/mol, 11 J/mol lower. `verified: false` means even the densest sampling left a phase with a positive driving force; treat that result with care. `verify: false` skips the check when speed matters more than certainty.

## Example 5.1 — one temperature

Cu–33Ni (at.%) at 1500 K.

```
{"db": "cuni_cost507", "composition": {"NI": 0.33}, "balance": "CU", "T": 1500}
```


```
{"alloy": {"NI": 0.33, "CU": 0.67},
 "units_in": "mole_fraction",
 "candidate_phases": ["FCC_A1", "LIQUID"],
 "results": {
   "T": 1500.0, "P": 101325.0, "converged": true,
   "GM_system": -87035.357,
   "chemical_potentials": {"CU": -86428.337, "NI": -88267.791},
   "phases": [
     {"phase": "LIQUID", "composition_set": 1, "state": null, "equivalent_to":
    null,
      "fraction": 0.534426,
      "composition": {"CU": 0.729662, "NI": 0.270338},
      "composition_wt": {"CU": 0.745054, "NI": 0.254946},
      "GM": -86925.612,
      "site_fractions": [{"CU": 0.729662, "NI": 0.270338}]},
    {"phase": "FCC_A1", "composition_set": 1, "state": null, "equivalent_to":
    null,
      "fraction": 0.465574,
      "composition": {"CU": 0.601515, "NI": 0.398485},
      "composition_wt": {"CU": 0.620407, "NI": 0.379593},
      "GM": -87161.332,
      "site_fractions": [{"CU": 0.601515, "NI": 0.398485}, {"VA": 1.0}]}],
   "stability_check": {"verified": true, "sampling_density": "default",
                       "driving_forces_absent_phases": {}}}}
```

(Numbers rounded to 6 significant digits here; the server returns full precision.) The two `composition` values are the tie-line ends at 1500 K, and the lever rule holds: $0.5344 \times 0.2703 + 0.4656 \times 0.3985 = 0.33$.

## Example 5.2 — temperature sweep: solidus and liquidus

Ni–10Al from 1700 to 1725 K in one request.

```
r = s.equilibrium({"AL": 0.10}, [1700, 1717, 1718, 1719, 1720, 1725], balance="NI")
for res in r["results"]:
    print(res["T"], [(p["equivalent_to"] or p["phase"], round(p["fraction"], 4),
                      round(p["composition"]["AL"], 5)) for p in res["phases"]])
```

| T (K) | $\gamma$ fraction | $\gamma$ $x_{\mathrm{Al}}$ | Liquid fraction | Liquid $x_{\mathrm{Al}}$ |
|---|---|---|---|---|
| 1700 | 1.000 | 0.10000 | — | — |
| 1717 | 1.000 | 0.10000 | — | — |
| 1718 | 0.934 | 0.09937 | 0.066 | 0.10890 |
| 1719 | 0.513 | 0.09568 | 0.487 | 0.10456 |
| 1720 | — | — | 1.000 | 0.10000 |
| 1725 | — | — | 1.000 | 0.10000 |

So the solidus of Ni–10Al lies between 1717 and 1718 K and the liquidus between 1719 and 1720 K (the demo in section 11 interpolates them as 1717.82 K and 1719.98 K). $\gamma$ is reported as `FCC_L12` with `state: "disordered"` and `equivalent_to: "FCC_A1"`. Each temperature is solved and checked separately; the list is a convenience, not a speed-up.

## Example 5.3 — weight percent

Cu–30Ni by weight at 1500 K:

```
{"db": "cuni_cost507", "composition": {"NI": 30}, "units": "weight_percent", "balance": "CU", "T":
    1500}
```

The response's `alloy` is `{"NI": 0.316955, "CU": 0.683045}`. Liquid is 0.6362 of the atoms and FCC 0.3638. The phase compositions are the same tie-line ends as in 5.1, since the temperature is the same: liquid 25.49 wt% Ni and FCC 37.96 wt% Ni in `composition_wt`.

## Example 5.4 — restricting phases: metastable equilibrium

Ni–25Al at 1000 K is single-phase $\gamma'$. Leaving $\gamma'$ out gives the metastable equilibrium without it, useful when $\gamma'$ is kinetically suppressed, for example in rapid solidification.

```
{"db": "alni_dupin_2001", "composition": {"AL": 0.25}, "balance": "NI", "T": 1000,
 "phases": ["FCC_A1", "BCC_B2", "LIQUID"]}
```

| Request | Result |
|---|---|
| All phases | `FCC_L12` ordered ($\gamma'$), fraction 1.000 |
| `["FCC_A1", "BCC_B2", "LIQUID"]` | `FCC_A1` ($\gamma$) 0.520 at $x_{\mathrm{Al}}$ 0.1939 + `BCC_B2` ordered 0.480 at $x_{\mathrm{Al}}$ 0.3109 |

Name the disordered phase (`FCC_A1`) to exclude ordering; naming the ordered one (`FCC_L12`) allows both states.

## Example 5.5 — ternary with ordered phases

Ni–20Al–10Cr (at.%) at 1600 K in the `alcrni` database:

```
{"db": "alcrni", "composition": {"AL": 0.2, "CR": 0.1}, "balance": "NI", "T": 1600}
```

| Phase | `state` → `equivalent_to` | Fraction | $x_{\mathrm{Al}}$ | $x_{\mathrm{Cr}}$ | $x_{\mathrm{Ni}}$ | GM (J/mol) |
|---|---|---|---|---|---|---|
| `L12_FCC` | disordered → `FCC_A1` | 0.8555 | 0.1857 | 0.1073 | 0.7070 | −116 858.1 |
| `B2` | ordered → `B2` | 0.1445 | 0.2848 | 0.0567 | 0.6585 | −125 959.1 |

`GM_system` is −118 173.48 J/mol and `stability_check` shows `"sampling_density": 2000` with a liquid driving force of −26.09 J/mol. B2's `site_fractions` show its ordering: Al fills 0.567 of the first sublattice but 0.002 of the second.

## Example 5.6 — one phase twice: $\gamma + \gamma'$ and a miscibility gap

Ni–20Al at 1000 K returns `FCC_L12` twice:

| Phase | `composition_set` | `state` → `equivalent_to` | Fraction | $x_{\mathrm{Al}}$ |
|---|---|---|---|---|
| `FCC_L12` | 1 | ordered → `FCC_L12` ($\gamma'$) | 0.6697 | 0.2372 |
| `FCC_L12` | 2 | disordered → `FCC_A1` ($\gamma$) | 0.3303 | 0.1246 |

Ni–40Al–20Cr at 1600 K (`alcrni`) does the same with B2: ordered B2 0.8268 ($x_{\mathrm{Al}}$ 0.4642) and disordered B2, which is Cr-rich $\alpha$ (`BCC_A2`, $x_{\mathrm{Cr}}$ 0.8811), 0.1732. Group results by `equivalent_to or phase` to get physical phase names:

```python
r = s.equilibrium({"AL": 0.20}, 1000, balance="NI")["results"]
for p in r["phases"]:
    print(p["equivalent_to"] or p["phase"], round(p["fraction"], 4), round(p["composition"]["AL"], 4))
# FCC_L12 0.6697 0.2372
# FCC_A1 0.3303 0.1246
```

# 6. API 2: Phase energies — `POST /phase_energies`

For one alloy composition and temperature, API 2 gives the Gibbs energy of every candidate phase at that composition, stable or not, with its own chemical potentials and its driving force. Use it for metastability, nucleation driving forces and free-energy inputs at a known composition.

| **Access** | **Call** |
|---|---|
| HTTP | `POST /phase_energies` |
| Python, in-process | `CalphadSystem.phase_energies(composition, T, P=101325, phases=None, units="mole_fraction", balance=None)` |
| Python client | `CalphadClient.phase_energies(composition, T, P=None, phases=None, units=None, balance=None, db=None)` |
| C++ client | `CalphadClient::phase_energies(composition, T, balance, extra)` |

## What it computes

Each phase is minimised on its own, as one homogeneous phase at the alloy composition. Its internal variables (sublattice occupancies, ordering) relax; its composition does not. pycalphad's `IsolatedPhase` does this. The framework also refines each phase from ordered and disordered starting configurations and keeps the lowest, because pycalphad alone returned disordered $\gamma'$ for Ni–25Al at 1600 K, 355 J/mol too high.

## Request

The alloy fields of section 3, plus `db` and `T` (a single number, K). The equilibrium used for comparison is always verified.

## Response

| **Field** | **Meaning** |
|---|---|
| `alloy`, `T`, `P` | Conditions |
| `GM_equilibrium` | G of the equilibrium mixture at this composition, J/mol |
| `stable_phases` | Equilibrium phases; repeats as `"FCC_L12#2"` |
| `stability_check` | As in API 1 |
| `phases[]` | One entry per candidate phase, lowest `GM` first, unreachable phases last |

Each entry of `phases[]`:

| **Field** | **Meaning** |
|---|---|
| `GM` | G of this phase alone at the alloy composition, J/mol; `null` if it cannot reach it |
| `chemical_potentials` | This phase's tangent plane at this composition, J/mol per element |

| Field | Meaning |
|---|---|
| `state`, `equivalent_to`, `site_fractions` | As in API 1, for this phase's minimised state |
| `GM_above_equilibrium` | `GM` − `GM_equilibrium`, $\geq 0$: the cost of forcing the whole alloy into this phase |
| `driving_force` | J/mol, against the equilibrium chemical potentials: 0 for stable phases, negative for the rest |
| `splits_if_allowed` | Present when this phase on its own would rather separate into two compositions: `GM` of that split and its `composition_sets` |
| `note` | Why `GM` is `null` |

**`GM_above_equilibrium` versus `driving_force`.** The first is measured at the alloy composition. The second is the largest gap between the equilibrium tangent plane and the phase's G curve over all the phase's compositions. A stable phase has `driving_force` 0 even when its `GM_above_equilibrium` is positive, because a different composition of it touches the tangent. Use `driving_force` for nucleation.

**Diffusion potentials.** In a binary A–B, `chemical_potentials[B]` − `chemical_potentials[A]` of a phase is dG/dc of that phase at this composition: the diffusion potential a KKS solver equates between phases.

## Example 6.1 — a two-phase alloy

Cu–33Ni at 1500 K, where liquid and FCC coexist:

```
{"db": "cuni_cost507", "composition": {"NI": 0.33}, "balance": "CU", "T": 1500}
```


```
{"alloy": {"NI": 0.33, "CU": 0.67}, "T": 1500.0, "P": 101325.0,
 "GM_equilibrium": -87035.357,
 "stable_phases": ["LIQUID", "FCC_A1"],
 "stability_check": {"verified": true, "sampling_density": "default", "driving_forces_absent_phases":
    {}},
 "phases": [
   {"phase": "LIQUID", "GM": -86975.228,
    "chemical_potentials": {"CU": -87010.056, "NI": -86904.515},
    "state": null, "equivalent_to": null,
    "site_fractions": [{"CU": 0.67, "NI": 0.33}],
    "GM_above_equilibrium": 60.129, "driving_force": 0.0},
   {"phase": "FCC_A1", "GM": -86970.388,
    "chemical_potentials": {"CU": -85719.159, "NI": -89510.761},
    "state": null, "equivalent_to": null,
    "site_fractions": [{"CU": 0.67, "NI": 0.33}, {"VA": 1.0}],
    "GM_above_equilibrium": 64.969, "driving_force": 0.0}]}
```


Forcing the alloy to be all liquid costs 60.1 J/mol and all FCC 65.0 J/mol; the two-phase mixture is lower than either. The diffusion potentials at $x_{\mathrm{Ni}} = 0.33$ are 105.5 J/mol in the liquid and −3791.6 J/mol in FCC. They differ because $x_{\mathrm{Ni}} = 0.33$ is not on either tie-line end.

## Example 6.2 — an ordered phase and compounds

Ni–25Al at 1600 K (`alni_dupin_2001`):

| Phase | `state` | GM (J/mol) | Above equilibrium (J/mol) | Driving force (J/mol) |
|---|---|---|---|---|
| `FCC_L12` | ordered ($\gamma'$) | −122 316.2 | 0.0 | 0.0 |
| `BCC_B2` | ordered | −122 024.5 | 291.7 | −98.1 |
| `LIQUID` | — | −121 793.5 | 522.7 | −522.4 |
| `AL3NI1`, `AL3NI2`, `AL3NI5` | — | `null` | — | — |

$\gamma'$ is stable. B2 at this composition is 291.7 J/mol above it, but a more Al-rich B2 comes within 98.1 J/mol of the tangent plane, which is the number that matters for whether B2 could nucleate. The three compounds are stoichiometric and cannot reach $x_{\mathrm{Al}} = 0.25$, so their `note` reads `phase cannot attain this composition (e.g. a stoichiometric compound)`.

## Example 6.3 — a phase that would rather split

Ni–20Al at 1000 K is $\gamma + \gamma'$, both described by `FCC_L12`. As one homogeneous phase, `FCC_L12` has GM = −74 334.4 J/mol, 122.7 J/mol above equilibrium. Allowed to separate, it splits into the equilibrium itself:


```
{"phase": "FCC_L12", "GM": -74334.4, "state": "ordered",
 "GM_above_equilibrium": 122.7, "driving_force": 0.0,
 "splits_if_allowed": {"GM": -74457.12,
   "composition_sets": [{"fraction": 0.6697, "composition": {"AL": 0.2372, "NI": 0.7628}},
                        {"fraction": 0.3303, "composition": {"AL": 0.1246, "NI": 0.8754}}]}}
```


`stable_phases` is `["FCC_L12", "FCC_L12#2"]`. The other phases are far away: B2 2476.1 J/mol above (driving force −2475.3) and liquid 8007.5 J/mol above (driving force −7624.4).

## Example 6.4 — ranking phases by driving force

```python
pe = s.phase_energies({"AL": 0.20}, 1000, balance="NI")
for p in pe["phases"]:
    if p["GM"] is None:
        continue
    print(f'{p["phase"]:8s} above eq {p["GM_above_equilibrium"]:8.1f}  driving force
     {p["driving_force"]:8.1f}')
# FCC_L12  above eq    122.7  driving force      0.0
# BCC_B2   above eq   2476.1  driving force  -2475.3
# LIQUID   above eq   8007.5  driving force  -7624.4
```

# 7. API 3: Energy surfaces — `POST /energy_surface`

At one temperature, API 3 returns G(x) of every candidate phase over the whole composition space: curves for a binary, surfaces for a ternary. Use it to plot free-energy diagrams, build lookup tables or fit your own free-energy functions.

| Access | Call |
|---|---|
| HTTP | `POST /energy_surface` |
| Python, in-process | `CalphadSystem.energy_surface(T, P=101325, elements=None, phases=None, step=0.02, mode="grid", pdens=500, max_points=200000)` |
| Python client | `CalphadClient.energy_surface(T, P=None, elements=None, phases=None, mode=None, step=None, pdens=None, max_points=None, db=None)` |
| C++ client | `CalphadClient::energy_surface(T, extra)` |

## Request

| Field | Type | Default | Meaning |
|---|---|---|---|
| `db` | string | the only database | Which database |
| `T` | number | required | K |
| `P` | number | 101325 | Pa |
| `elements` | list | all elements | A subsystem, for example `["AL", "NI"]` of Al–Cr–Ni |
| `phases` | list | all | Restrict the phases |
| `mode` | `"grid"` or `"points"` | `"grid"` | See below |
| `step` | number, $0 <$ step $\leq 0.5$ | 0.02 | Grid spacing in mole fraction |
| `pdens` | integer | 500 | Sampling density for phases with sublattices |
| `max_points` | integer | 200 000 | Upper limit on points per phase |

## The two modes

**`grid`** evaluates each phase on a regular composition grid that includes the pure elements. A binary at `step` 0.01 has 101 nodes; a ternary has 1326 at 0.02 and 5151 at 0.01.

- Substitutional phases (all elements on one sublattice, any other sublattice vacancies only: `LIQUID`, `FCC_A1`, `BCC_A2` and the like) get the exact G at each node: `"approximate": false`.

- Phases with sublattices (ordered phases, compounds) have internal configurations as well as a composition. pycalphad samples them and the lowest-energy sample in each grid cell is kept: `"approximate": true`. Its `X` is that sample's own composition, within half a step of the node, not the node itself. Raise `pdens` for more samples.
- Stoichiometric compounds only cover the cells they can reach, so their `n_points` is small.

**`points`** returns every sampled configuration, including higher-energy orderings, thinned at random to `max_points` if needed. Use it to see all branches of an ordered phase.

## Response

| Field | Meaning |
|---|---|
| `T`, `P` | Conditions |
| `elements` | Element order of the columns of `X` (alphabetical) |
| `mode`, `step` | As requested (`step` is `null` in points mode) |
| `phases[].phase` | Phase name |
| `phases[].n_points` | Number of points |
| `phases[].approximate` | `true` for sampled phases in grid mode |
| `phases[].X` | List of compositions; each row is mole fractions in `elements` order |
| `phases[].GM` | G at each row of `X`, J/mol |

## Example 7.1 — binary free-energy curves

Cu–Ni at 1500 K: 101 exact points for each of `FCC_A1` and `LIQUID`.

```
{"db": "cuni_cost507", "T": 1500, "step": 0.01}
```

The first rows of `FCC_A1` are `X: [[0.0, 1.0], [0.01, 0.99], ...]` (columns Cu, Ni, so pure Ni first) and `GM: [-81682.09, -82248.35, ...]`. Plotting:

```
import numpy as np
import matplotlib.pyplot as plt

surf = s.energy_surface(T=1500, step=0.01)      # or c.energy_surface(T=1500, step=0.01,
    db="cuni_cost507")
i_ni = surf["elements"].index("NI")
for ph in surf["phases"]:
    X = np.array(ph["X"])
    G = np.array(ph["GM"], dtype=float)
    order = np.argsort(X[:, i_ni])
    plt.plot(X[order, i_ni], G[order] / 1000, label=ph["phase"])
plt.xlabel("x_Ni")
plt.ylabel("G (kJ/mol)")
plt.legend()
plt.show()
```

## Example 7.2 — ternary surface on a Gibbs triangle

Al–Cr–Ni at 1273 K, `step` 0.02: 1326 exact points for `LIQUID`; `B2` and `L12_FCC` are marked approximate.

```
surf = t.energy_surface(T=1273, step=0.02)        # t = CalphadSystem.from_file("alcrni.tdb")
el = surf["elements"]                              # ['AL', 'CR', 'NI']
liq = next(p for p in surf["phases"] if p["phase"] == "LIQUID")
X = np.array(liq["X"])
G = np.array(liq["GM"], dtype=float)
xa, xc = X[:, el.index("AL")], X[:, el.index("CR")]
px = xc + 0.5 * xa                                 # Ni bottom-left, Cr bottom-right, Al top
py = np.sqrt(3) / 2 * xa
plt.tricontourf(px, py, G / 1000, levels=20)
plt.colorbar(label="G of LIQUID (kJ/mol)")
plt.gca().set_aspect("equal")
plt.axis("off")
plt.show()
```

## Example 7.3 — a binary subsystem of a larger database

```
{"db": "alcrni", "T": 1273, "elements": ["AL", "NI"], "step": 0.02}
```

Returns 51 points per phase along the Al–Ni edge; Cr is left out of the calculation entirely.

## Example 7.4 — the lowest single phase at each composition

Because sampled phases are not exactly on the nodes, compare phases by grid cell, not by exact composition:

```
step = 0.02
surf = s.energy_surface(T=1000, step=step, phases=["FCC_L12", "BCC_B2", "LIQUID"])   # Ni-Al
i_al = surf["elements"].index("AL")
lowest = {}                                        # grid cell -> (phase, G)
for ph in surf["phases"]:
    for x, g in zip(ph["X"], ph["GM"]):
        cell = round(x[i_al] / step)
        if g is not None and (cell not in lowest or g < lowest[cell][1]):
            lowest[cell] = (ph["phase"], g)
for cell in range(0, 46, 5):
    print(f"x_Al = {cell * step:.2f}: {lowest[cell][0]}")
# x_Al = 0.00 ... 0.30: FCC_L12    x_Al = 0.40 ... 0.80: BCC_B2    x_Al = 0.90: LIQUID
```

This compares single phases, not mixtures; for which phases coexist, use API 1. Within one cell, a sampled phase sits up to half a step from the node, so near the pure-element ends, where G is steep, use API 2 for an exact comparison at one composition.

## Example 7.5 — every sampled configuration

```
{"db": "alni_dupin_2001", "T": 1000, "mode": "points", "pdens": 200, "phases": ["FCC_L12", "LIQUID"]}
```

Returns 1604 points for `FCC_L12` (ordered and disordered configurations together) and 402 for `LIQUID`. Plotted as a scatter of G against $x_{\mathrm{Al}}$, the lower edge of the `FCC_L12` cloud is its stable branch.

### Speed

Grid mode is fast: Cu–Ni at `step` 0.01 takes a few milliseconds, Al–Cr–Ni at `step` 0.05 about 0.07 s. Responses grow with the number of points, so prefer `step` $\geq$ 0.01 for ternaries over HTTP.

## 8. KKS module — `calphad_kks.py`

`KKSModel` turns a TDB into what a KKS phase-field solver needs for a binary alloy with two phases: each phase's free energy, the equilibrium tie lines, the KKS composition split at every grid point, and the driving force. Inputs are NumPy arrays of any shape, and T may be a field.

### 8.1 The model in brief

At every point, the overall composition c is split into the two phase compositions $c_a$ and $c_b$ so that both the mixture rule and equal diffusion potential hold:

$$c = h(\phi)\, c_a + \Big(1 - h(\phi)\Big)\, c_b, \qquad \frac{\partial f_a}{\partial c}(c_a) = \frac{\partial f_b}{\partial c}(c_b) = \tilde{\mu}$$

The phase field and the composition then evolve as:

$$\frac{\partial \phi}{\partial t} = M\Big[\varepsilon^2 \nabla^2 \phi - W g'(\phi) - h'(\phi)\,\Big(f_a(c_a) - f_b(c_b) - \tilde{\mu}\,(c_a - c_b)\Big)\Big]$$

$$\frac{\partial c}{\partial t} = \nabla \cdot \Big[D(\phi)\Big(h\,\nabla c_a + (1-h)\,\nabla c_b\Big)\Big]$$

with $\mathrm{h}(\phi) = \phi^3(6\phi^2 - 15\phi + 10)$ and $\mathrm{g}(\phi) = \phi^2(1-\phi)^2$. $\phi = 1$ is `phase_a`, f = G / Vm in J/m$^3$, and c is the mole fraction of the solute. The interface energy $\sigma$ and width $2\lambda$ ($\phi$ from 0.1 to 0.9) set the gradient and barrier coefficients (Kim, Kim & Suzuki, Phys. Rev. E 60, 7186, 1999):

$$\sigma = \frac{\varepsilon\sqrt{W}}{3\sqrt{2}}, \qquad 2\lambda = \frac{2.2\sqrt{2}\,\varepsilon}{\sqrt{W}}$$

The module provides everything inside the square brackets except $\varepsilon^2\nabla^2\phi$, which is your solver's stencil. M and D($\phi$) are yours to choose.

### 8.2 Constructing a model

```
KKSModel(system, solute, phase_a, phase_b, mode="exact", Vm=None, c_alloy=None, T=None,
         solvent=None, reference=None, n_c=201, c_range=(1e-4, 1 - 1e-4), branches=None)

KKSModel.from_tdb(path, solute, phase_a, phase_b, **same_keywords)
```

| Parameter | Default | Meaning |
|---|---|---|
| `system` | required | A `CalphadSystem` (or use `from_tdb` with a file path) |
| `solute` | required | Element B; c = $x_B$ |
| `phase_a` | required | Phase at $\phi = 1$, usually the solid (`"FCC_A1"`) |
| `phase_b` | required | Phase at $\phi = 0$, usually `"LIQUID"` |
| `mode` | `"exact"` | `"exact"`, `"table"` or `"parabolic"` (8.3) |
| `Vm` | `None` | Molar volume, $m^3$/mol; needed for anything per volume |
| `c_alloy` | `None` | Nominal composition; picks the right two-phase field when the two phases meet more than once |
| `T` | `None` | Temperatures for `table` and `parabolic`: a list, or a tuple `(Tmin, Tmax, step)` |
| `solvent` | the other element | Element A; required when the database has more than two elements |
| `reference` | exact where possible | What tie lines and parabolas are computed from: `"exact"` or `"table"` |
| `n_c`, `c_range` | 201, ($10^{-4}$, 1 − $10^{-4}$) | Composition grid for tabulated phases |
| `branches` | `None` | For order/disorder phases: `{"FCC_L12": "ordered"}`; options `"lowest"`, `"ordered"`, `"disordered"` |

After construction, check `kks.warnings` (a list of strings) and the `consistent` flag of the tie lines. Other attributes: `phases` (the pair of free-energy objects in use), `reference`, `mode`, `Vm`, `T_valid` (temperature range covered), `phase_names`, `A` (solvent) and `B` (solute).

| mode | What it is | Phases | Accuracy | split on 512×512 | Needs T |
|---|---|---|---|---|---|

## 8.3 Free-energy modes

| mode | What it is | Phases | Accuracy | split on 512×512 | Needs T |
|---|---|---|---|---|---|
| exact | The CALPHAD expression compiled to machine code, exact 1st and 2nd derivatives | Substitutional only (liquid, $\gamma$, $\alpha$) | Exact | 0.22 s cold, 0.06 s warm-started | No |
| table | CALPHAD G on a (c, T) grid, splined; ideal mixing added back analytically | Any, including ordered $\gamma'$ | Within about $10^{-3}$ J/mol of exact | 1.1 s | Yes |
| parabolic | $G_{\mathrm{eq}} + \mu_{\mathrm{eq}}(\mathrm{c} - c_{\mathrm{eq}}) + \mathrm{A}/2\ (\mathrm{c} - c_{\mathrm{eq}})^2$ about each tie line, coefficients linear in T | Any | Exact on the tie line; driving force about 0.5% off at 0.03 from it | 0.03 s, closed form | Yes |

Start with `exact` for liquid and $\gamma$. Use `parabolic` for speed in large 2-D and 3-D runs. Use `table` for ordered phases, which `exact` refuses. Each free-energy object can be called directly: `G, dG, d2G = kks.phases[0](c, T)`.

Order/disorder phases in `table` and `parabolic` mode take a `branches` entry. `"lowest"` (the default) uses whichever state is lower at each point, so G has a kink where they swap, and `kks.warnings` says so. `"ordered"` follows the ordered branch even where it is metastable, which gives a smooth G suited to KKS; `"disordered"` follows the other.

## 8.4 Methods

| Method | Returns |
|---|---|
| `split(c, phi=None, T=None, h=None, ca0=None, cb0=None, tol=1e-10, maxit=60, return_info=False)` | `ca, cb, mu` (and an info dict): the KKS split at every point. Give `phi` or `h` directly. `ca0`, `cb0` warm-start Newton from the previous step |
| `driving_force(ca, cb, mu, T, per_volume=True)` | $f_a(c_a) - f_b(c_b) - \mu(c_a - c_b)$: J/m$^3$, or J/mol with `per_volume=False`. Negative favours `phase_a` |

| Method | Returns |
|---|---|
| `free_energy_density(c, phi, T, per_volume=True)` | Chemical part h $f_a$ + (1 − h) $f_b$, without the double well or gradient terms |
| `dmu_dc(ca, cb, phi, T)` | d$\mu$/dc at fixed $\phi = 1$ / (h/$f_a''$ + (1 − h)/$f_b''$), J/mol, for the $\mu$-form of the diffusion equation |
| `energies(ca, cb, T)` | `(G_a, dG_a, d2G_a), (G_b, dG_b, d2G_b)`, J/mol |
| `tie_lines(temps)`, `tie_line(T)` | Equilibrium data per temperature (table below); `None` where the phases do not coexist |
| `h`, `dh`, `g`, `dg` | Interpolation and double-well functions and their derivatives (static) |
| `interface_parameters(sigma, width)` | `(eps2, W)` from $\sigma$ in J/m$^2$ and $2\lambda$ in m (static) |
| `parameters(temps, c_grid=None, include_tables=False)` | Everything above as a JSON-ready dict |
| `export_parabolic_csv(path, temps)` | Writes the CSV that `cpp/kks_parabolic.hpp` reads; returns the row count |
| `export_json(path, temps, include_tables=True, c_grid=None)` | Writes `parameters(...)` to a file |

Each tie line is a dict:

| Key | Meaning |
|---|---|
| `T` | K |
| `c_a`, `c_b` | Equilibrium compositions of `phase_a` and `phase_b` (common tangent of the model's energies) |
| `k` | Partition coefficient $c_a$ / $c_b$ |
| `mu` | Equilibrium diffusion potential, J/mol |
| `G_a`, `G_b` | G of each phase at its tie-line composition, J/mol |
| `A_a`, `A_b` | Curvatures d$^2$G/dc$^2$ there, J/mol |
| `c_a_pycalphad`, `c_b_pycalphad` | The same compositions from pycalphad's own equilibrium |
| `consistent` | `true` when the two agree within $10^{-3}$ |

`split` uses damped Newton iterations (exact, table) or a closed form (parabolic). With `return_info=True` the info dict holds `iterations`, `max_residual` and, if it did not converge, a `warning`.

## Example 8.1 — $\gamma$/liquid tie lines in Ni–Al

```
from calphad_kks import KKSModel

kks = KKSModel.from_tdb("alni_dupin_2001.tdb", solute="AL", phase_a="FCC_A1", phase_b="LIQUID",
                        mode="exact", Vm=7.0e-6, c_alloy=0.10)
for r in kks.tie_lines([1700, 1710, 1715]):
    print(r["T"], r["c_a"], r["c_b"], r["k"], r["A_a"], r["A_b"], r["consistent"])
```

| T (K) | $c_\gamma$ | $c_L$ | k | $\mu$ (J/mol) | $A_\gamma$ (J/mol) | $A_L$ (J/mol) | Consistent |
|---|---|---|---|---|---|---|---|
| 1700 | 0.143764 | 0.161649 | 0.8894 | −120 105.1 | 336 664 | 283 245 | yes |
| 1710 | 0.122723 | 0.136548 | 0.8988 | −126 973.7 | 329 261 | 276 972 | yes |
| 1715 | 0.109193 | 0.120494 | 0.9062 | −131 340.3 | 326 705 | 275 600 | yes |

Building the exact model takes 0.04 s once the database is loaded.

## Example 8.2 — split and driving force

Three points at 1710 K: inside $\gamma$, inside the interface, and inside the liquid.

```
import numpy as np

c   = np.array([0.10, 0.13, 0.13])
phi = np.array([1.0,  0.5,  0.0])
ca, cb, mu, info = kks.split(c, phi=phi, T=1710.0, return_info=True)
dF = kks.driving_force(ca, cb, mu, 1710.0)       # J/m^3
```

| c | $\phi$ | $c_\gamma$ | $c_L$ | $\mu$ (J/mol) | Driving force (J/m$^3$) |
|---|---|---|---|---|---|
| 0.10 | 1.0 | 0.10000 | 0.10955 | −134 411.0 | $-1.241 \times 10^7$ |
| 0.13 | 0.5 | 0.12306 | 0.13694 | −126 864.1 | $+2.17 \times 10^5$ |
| 0.13 | 0.0 | 0.11722 | 0.13000 | −128 783.5 | $-3.44 \times 10^6$ |

`info` is `{"iterations": 4, "max_residual": 8.9e-17}`. Where $\phi = 1$, $c_\gamma$ equals c; where $\phi = 0$, $c_L$ equals c; the other composition is the one that would be in equilibrium with it. A negative driving force pushes $\phi$ toward 1 ($\gamma$ grows): a liquid at $x_{\mathrm{Al}} = 0.13$, below $c_L = 0.1365$, is undercooled. `kks.dmu_dc(ca, cb, phi, 1710.0)` gives 326 570, 300 946 and 275 822 J/mol at the three points.

## Example 8.3 — parabolic energies for speed

```
kp = KKSModel.from_tdb("alni_dupin_2001.tdb", solute="AL", phase_a="FCC_A1", phase_b="LIQUID",
                       mode="parabolic", Vm=7.0e-6, c_alloy=0.10, T=(1690, 1725, 1))
kp.T_valid                                  # (1690.0, 1725.0)
c_eq, A, mu_eq, G_eq = kp.phases[0].coefficients(np.array([1710.0]))
# c_eq 0.12272, A 329 260.9, mu_eq -126 973.72, G_eq -117 580.03
ca, cb, mu = kp.split(np.array([0.13]), phi=0.5, T=1710.0)
# 0.123056, 0.136944, -126 864.12  (exact model: 0.12306, 0.13694, -126 864.1)
```

Building it takes about 4.5 s for 36 temperatures (one exact tie line each). Outside `T_valid` the coefficients are held at the nearest end.

## Example 8.4 — a temperature field (additive manufacturing)

Pass T as an array the same shape as c. A frozen-temperature gradient moving at the scan speed:

```
T = T_solidus + G_thermal * (x - x0 - V * t)          # array, K
ca, cb, mu = kks.split(c, phi=phi, T=T, ca0=ca, cb0=cb)   # warm start from the last step
dF = kks.driving_force(ca, cb, mu, T)
```

On a $512 \times 512$ grid with T varying over 20 K, the exact split took 0.22 s from a cold start and 0.06 s warm-started; mass balance held to $10^{-16}$ and the diffusion potentials agreed to $10^{-8}$ J/mol.

## Example 8.5 — table mode

```
kt = KKSModel.from_tdb("alni_dupin_2001.tdb", solute="AL", phase_a="FCC_A1", phase_b="LIQUID",
                       mode="table", Vm=7.0e-6, c_alloy=0.10, T=(1690, 1725, 5))
kt.split(np.array([0.13]), phi=0.5, T=1710.0)    # 0.1230559, 0.1369441, -126 864.10
```

For substitutional phases the table is filled from the exact expression and builds in 0.03 s. Its value is for phases `exact` refuses.

## Example 8.6 — $\gamma'$/liquid with an ordered phase

```
kg = KKSModel.from_tdb("alni_dupin_2001.tdb", solute="AL", phase_a="FCC_L12", phase_b="LIQUID",
                       mode="parabolic", Vm=7.0e-6, c_alloy=0.32, T=(1580, 1640, 5),
                       branches={"FCC_L12": "ordered"}, n_c=101)
```

| T (K) | $c_{\gamma'}$ | $c_L$ | k | $A_{\gamma'}$ (J/mol) | $A_L$ (J/mol) | Consistent |
|---|---|---|---|---|---|---|
| 1600 | 0.31238 | 0.34392 | 0.9083 | 514 540 | 363 120 | yes |
| 1620 | 0.29517 | 0.31802 | 0.9282 | 523 118 | 351 893 | yes |

Building it takes about 14 s, because each tabulated point minimises $\gamma'$ from several starting configurations. Without `branches`, `kks.warnings` holds: `FCC_L12 switches between ordered and disordered states inside the tabulated range, so G has a kink there. Pass branches={'FCC_L12': 'ordered'} (or use the disordered phase, e.g. FCC_A1).` Asking for `mode="exact"` with `FCC_L12` raises `CalphadError: FCC_L12 has sublattices or vacancies mixing with the elements, so G at fixed composition needs an internal minimisation. Use mode='table'.`

## Example 8.7 — a complete solver loop (1-D, explicit)

Condensed from `examples/kks_1d_directional.py`:

```
eps2, W = KKSModel.interface_parameters(sigma=0.3, width=5 * dx)   # J/m, J/m^3
M = DL / eps2                                                      # numerical choice
dt = 0.2 * dx**2 / DL
ca, cb = c.copy(), c.copy()
for n in range(nsteps):
    T = T_sol + G_thermal * (x - x0 - V * n * dt)
    h = kks.h(phi)
    ca, cb, mu = kks.split(c, phi=phi, T=T, ca0=ca, cb0=cb)
    dF = kks.driving_force(ca, cb, mu, T)

    lap = np.zeros_like(phi)
    lap[1:-1] = (phi[2:] - 2 * phi[1:-1] + phi[:-2]) / dx**2
    dphi = M * (eps2 * lap - W * kks.dg(phi) - kks.dh(phi) * dF)

    hf = 0.5 * (h[1:] + h[:-1])                          # face values
    D = hf * DS + (1 - hf) * DL
    flux = -D * (hf * np.diff(ca) + (1 - hf) * np.diff(cb)) / dx
    dc = np.zeros_like(c)
    dc[:-1] -= flux / dx
    dc[1:] += flux / dx

    phi = np.clip(phi + dt * dphi, 0.0, 1.0)
    c = c + dt * dc
```

The flux form conserves solute to round-off. For $\sigma = 0.3$ J/m$^2$ and $2\lambda = 25$ nm, `interface_parameters` gives $\varepsilon^2 = 1.023 \times 10^{-8}$ J/m and $W = 1.584 \times 10^{8}$ J/m$^3$.

## Example 8.8 — exporting for another code

```
kp.export_parabolic_csv("nial_gamma_liquid.csv", np.arange(1700, 1721, 5.0))   # returns 5
```

```
# solute=AL solvent=NI phase_a=FCC_A1 phase_b=LIQUID Vm=7e-06
T_K,c_a_eq,c_b_eq,mu_eq,G_a_eq,G_b_eq,A_a,A_b
1700.000000,0.143763542452,0.161649248867,-120105.116106,-119340.612623,-121488.777469,336663.731971,
    283245.196776
1705.000000,0.133994065231,0.149977523559,-123310.17763,-118570.432255,-120541.35534,332850.034599,
    279860.081806
1710.000000,0.122723020193,0.136548406812,-126973.718923,-117580.030144,-119335.490898,329260.946062,
    276972.440606
1715.000000,0.109193169883,0.120493579965,-131340.285569,-116254.025014,-117738.224101,326705.352428,
    275599.576794
1720.000000,0.0917113337238,0.0999041407631,-136985.326052,-114331.778538,-115454.072881,328151.734925
    ,279293.624122
```

`kp.export_json("nial.json", temps, include_tables=True)` writes tie lines plus G, dG/dc and d$^2$G/dc$^2$ of both phases on a composition grid at each temperature, for codes that interpolate tables themselves.

## What the module does not do

It handles two phases and two elements. The phase-field mobility M, an anti-trapping current and the thin-interface calibration of M belong in your solver; without them, a thick interface traps solute when the front runs fast (the demo in section 11 uses a numerical M).

# 9. KKS over HTTP and in C++

`POST /kks/parameters` gives any language the tie lines, parabolic coefficients and, optionally, free-energy tables of a KKS model. `cpp/kks_parabolic.hpp` then runs the split and the driving force inside a C++ solver with no further calls to the server.

## 9.1 `POST /kks/parameters`

| Field | Type | Default | Meaning |
|---|---|---|---|
| `db` | string | the only database | Which database |
| `solute` | string | required | c = mole fraction of this element |
| `solvent` | string | the other element | Required when the database has more than two elements |
| `phase_a` | string | required | Phase at $\phi = 1$ |
| `phase_b` | string | required | Phase at $\phi = 0$ |
| `T` | list of numbers | required | Temperatures, K |
| `mode` | `"exact"`, `"table"`, `"parabolic"` | `"parabolic"` | Note: the Python class defaults to `"exact"` |
| `c_alloy` | number | none | Nominal composition; picks the two-phase field |
| `Vm` | number | none | $m^3$/mol, echoed back for the client's use |
| `branches` | object | none | `{"FCC_L12": "ordered"}` and the like |
| `include_tables` | boolean | `false` | Also return G, dG/dc and $d^2G/dc^2$ on a composition grid |
| `n_c` | integer, 8–2001 | 199 | Points in that grid, evenly spaced from 0.001 to 0.999 |

The response has `solute`, `solvent`, `phase_a`, `phase_b`, `mode`, `Vm`, a `convention` string, `tie_lines` (one dict per temperature as in 8.4, or `null` where the phases do not coexist), `warnings`, and, when asked, `tables`: `{"T": [...], "c": [...], "phases": {name: {"G": [[...]], "dG": [[...]], "d2G": [[...]]}}}` with each array indexed `[temperature][composition]`.

Request:

```
{"db": "alni_dupin_2001", "solute": "AL", "phase_a": "FCC_A1", "phase_b": "LIQUID",
 "T": [1710], "mode": "parabolic", "c_alloy": 0.10, "Vm": 7.0e-6}
```

Response:

```
{"solute": "AL", "solvent": "NI", "phase_a": "FCC_A1", "phase_b": "LIQUID",
 "mode": "parabolic", "Vm": 7e-06,
 "convention": "c = x_solute; phase_a at phi=1; energies J/mol; f_parabolic = G_eq + mu_eq (c - c_eq)
    + A/2 (c - c_eq)^2",
 "tie_lines": [{"T": 1710.0, "c_a": 0.122723, "c_b": 0.136548, "k": 0.898751,
                "mu": -126973.719, "G_a": -117580.030, "G_b": -119335.491,
                "A_a": 329260.946, "A_b": 276972.441,
```

```
              "c_a_pycalphad": 0.122723, "c_b_pycalphad": 0.136548, "consistent": true}],
 "warnings": []}
```

From Python:

```
p = c.kks_parameters("AL", "FCC_L12", "LIQUID", T=[1600, 1620], mode="table", c_alloy=0.32,
                     branches={"FCC_L12": "ordered"}, db="alni_dupin_2001")
[(r["T"], r["c_a"], r["c_b"]) for r in p["tie_lines"]]
# [(1600.0, 0.31238, 0.34392), (1620.0, 0.29517, 0.31802)]
```

The server keeps each model it builds, keyed on all the settings (and on `T` for `table` and `parabolic`). The $\gamma'$ model above takes about 14 s the first time and is instant on repeats until the server restarts. A tabulated ordered branch may cover only part of the composition range; in `tables`, values outside that range repeat the edge value.

## 9.2 `cpp/kks_parabolic.hpp`

A header-only struct with no dependencies. It stores the parabolic coefficients at a list of temperatures and interpolates them linearly in T, holding the end values outside the table, so T can be a field.

| Member | Meaning |
|---|---|
| `static KKSParabolic from_csv(path, Vm)` | Load a CSV written by `export_parabolic_csv` |
| `void add(T, ca, cb, mu, Ga, Gb, Aa, Ab)` | Add one temperature (rows must be added in rising T) |
| `Coef at(T)` | Interpolated coefficients `{ca, cb, mu, Ga, Gb, Aa, Ab}` |
| `void split(c, phi, T, double& ca, double& cb, double& mu)` | KKS split, closed form |
| `double driving_force(ca, cb, mu, T)` | J/m$^3$; throws if `Vm` $\leq 0$ |
| `double dmu_dc(phi, T)` | d$\mu$/dc at fixed $\phi$, J/mol |
| `static h, dh, g, dg` | Interpolation and double-well functions |
| `static f_a(c, coef), f_b(c, coef)` | Parabolic free energies, J/mol |
| `static interface_parameters(sigma, width, double& eps2, double& W)` | As in 8.4 |
| `std::vector<double> T, ca, cb, mu, Ga, Gb, Aa, Ab; double Vm` | The stored table |

## Example 9.1 — from a CSV exported in Python

```
#include <cstdio>
#include "kks_parabolic.hpp"

int main() {
    auto kks = calphad::KKSParabolic::from_csv("nial_gamma_liquid.csv", 7.0e-6);
    double ca, cb, mu;
    kks.split(0.12, 0.5, 1711.3, ca, cb, mu);
    double dF = kks.driving_force(ca, cb, mu, 1711.3);
    std::printf("%.6f %.6f %.2f %.5e %.1f\n", ca, cb, mu, dF, kks.dmu_dc(0.5, 1711.3));
```

```
}
// 0.113913 0.126087 -129848.10 -3.45117e+06 300373.7
```

```
g++ -std=c++17 -O2 -I path/to/calphad_server/cpp my_solver.cpp -o my_solver
```

The Python parabolic model gives the same numbers to every printed digit at this interpolated temperature.

## Example 9.2 — coefficients straight from the server

`cpp/kks_example.cpp` does this; the core of it:

```
calphad::CalphadClient c("127.0.0.1", 8000, "alni_dupin_2001");
json body = {{"solute", "AL"}, {"phase_a", "FCC_A1"}, {"phase_b", "LIQUID"},
             {"mode", "parabolic"}, {"c_alloy", 0.10}, {"Vm", 7.0e-6}};
std::vector<double> temps;
for (double T = 1700; T <= 1720; T += 2.5) temps.push_back(T);
body["T"] = temps;
json p = c.kks_parameters(body);                // once, at start-up

calphad::KKSParabolic kks;
kks.Vm = 7.0e-6;
for (auto& r : p["tie_lines"])
    if (!r.is_null())
        kks.add(r["T"], r["c_a"], r["c_b"], r["mu"], r["G_a"], r["G_b"], r["A_a"], r["A_b"]);
```

Its output begins:

```
loaded 9 tie lines, 1700-1720 K
T=1710 K: c_gamma=0.12272 c_L=0.13655 k=0.8988 A_gamma=329261 A_L=276972 J/mol
  c=0.10 phi=0.0 -> c_a=0.09198 c_b=0.10000 mu=-137096.6 J/mol dF=-1.5797e+07 J/m^3
```

## Example 9.3 — inside a C++ time loop

```
for (size_t i = 0; i < n; ++i) {
    double ca, cb, mu;
    kks.split(c[i], phi[i], T[i], ca, cb, mu);
    c_a[i] = ca;
    c_b[i] = cb;
    dF[i] = kks.driving_force(ca, cb, mu, T[i]);
    dphi[i] = M * (eps2 * lap_phi[i] - W * calphad::KKSParabolic::dg(phi[i])
                   - calphad::KKSParabolic::dh(phi[i]) * dF[i]);
}
```

The loop is independent per point, so it parallelises directly with OpenMP or on a GPU port of the same formulas.

# 10. Clients and other languages

Any language that can send JSON over HTTP can use the server. Ready-made clients exist for Python and C++, and `/openapi.json` can generate typed clients for most other languages.

## 10.1 Python client — `calphad_client.py`

Standard library only, so the one file can be copied into any project.

```
CalphadClient(url="http://127.0.0.1:8000", db=None, timeout=600)
```

`db` sets a default database for every call; `timeout` is in seconds.

| Method | Endpoint |
|---|---|
| `health()` | `GET /health` |
| `databases()` | `GET /databases` |
| `info(db=None)` | `GET /databases/{db}` |
| `load_database(path=None, tdb_text=None, name=None)` | `POST /databases` |
| `upload_tdb(local_path, name=None)` | `POST /databases` with the file's contents |
| `equilibrium(composition, T, P=None, phases=None, units=None, balance=None, verify=None, db=None)` | `POST /equilibrium` |
| `phase_energies(composition, T, P=None, phases=None, units=None, balance=None, db=None)` | `POST /phase_energies` |
| `energy_surface(T, P=None, elements=None, phases=None, mode=None, step=None, pdens=None, max_points=None, db=None)` | `POST /energy_surface` |
| `kks_parameters(solute, phase_a, phase_b, T, mode=None, c_alloy=None, Vm=None, solvent=None, branches=None, include_tables=None, n_c=None, db=None)` | `POST /kks/parameters` |

Arguments left as `None` are not sent, so the server's defaults apply. Every method returns the parsed JSON as Python dicts and lists.

```
from calphad_client import CalphadClient, CalphadServerError

c = CalphadClient("http://127.0.0.1:8000", db="alni_dupin_2001")
try:
    r = c.equilibrium({"AL": 0.10}, T=[1715, 1718, 1720], balance="NI")
except CalphadServerError as e:
    print(e)            # e.g. HTTP 400: Unknown element(s) ['FE']. Database has ['AL', 'NI'].
```

`python calphad_client.py http://127.0.0.1:8000` prints the server's health, a quick connection check.

## 10.2 C++ client — `cpp/calphad_client.hpp`

Header-only. It needs `httplib.h` and `json.hpp`, which `cpp/CMakeLists.txt` downloads into `build/deps` on the first build.

```
calphad::CalphadClient(const std::string& host = "127.0.0.1", int port = 8000, const std::string& db
    = "");
```

| Method | Endpoint |
|---|---|
| `health()`, `databases()`, `info(db = "")` | The `GET` endpoints |
| `load_database_path(path, name = "")` | `POST /databases` |
| `equilibrium(composition, double T, balance = "", json extra = {})` | `POST /equilibrium` |
| `equilibrium(composition, std::vector<double> temps, balance = "", json extra = {})` | Several temperatures |
| `phase_energies(composition, double T, balance = "", json extra = {})` | `POST /phase_energies` |
| `energy_surface(double T, json extra = {})` | `POST /energy_surface` |
| `kks_parameters(json body)` | `POST /kks/parameters` |
| `get(path)`, `post(path, json body)` | Any endpoint |

`composition` is a `std::map<std::string, double>`. `extra` carries any other request field, such as `{{"units", "weight_percent"}}`, `{{"phases", {"FCC_A1", "LIQUID"}}}` or `{{"verify", false}}`. Results are `nlohmann::json`. Errors throw `std::runtime_error`: `calphad server unreachable (...)` when the server is down, or the HTTP status and detail.

```
#include <cstdio>
#include "calphad_client.hpp"

int main() {
    calphad::CalphadClient c("127.0.0.1", 8000, "cuni_cost507");
    calphad::json r = c.equilibrium({{"NI", 30.0}}, 1500.0, "CU", {{"units", "weight_percent"}});
    for (auto& ph : r["results"]["phases"])
        std::printf("%s %.4f\n", ph["phase"].get<std::string>().c_str(),
     ph["fraction"].get<double>());
    try {
        c.equilibrium({{"FE", 0.3}}, 1500.0, "CU");
    } catch (const std::exception& e) {
        std::printf("error: %s\n", e.what());
    }
}
// LIQUID 0.6362
// FCC_A1 0.3638
// error: HTTP 400 /equilibrium: "Unknown element(s) ['FE']. Database has ['CU', 'NI']."
```

In your own CMake project:

```
find_package(Threads REQUIRED)
add_executable(my_solver my_solver.cpp)
target_include_directories(my_solver PRIVATE path/to/calphad_server/cpp path/to/deps)
target_link_libraries(my_solver PRIVATE Threads::Threads)    # add ws2_32 on Windows
```

### 10.3 Other languages

The request and response bodies are the JSON shown in sections 4–9. Two sketches, not run here:

MATLAB:

```
opts = weboptions('MediaType', 'application/json', 'Timeout', 600);
req = struct('db', 'cuni_cost507', 'composition', struct('NI', 0.33), 'balance', 'CU', 'T', 1500);
r = webwrite('http://127.0.0.1:8000/equilibrium', req, opts);
r.results.phases(1).fraction
```

Julia (packages HTTP and JSON3):

```
using HTTP, JSON3
body = JSON3.write(Dict("db" => "cuni_cost507", "composition" => Dict("NI" => 0.33),
                        "balance" => "CU", "T" => 1500))
resp = HTTP.post("http://127.0.0.1:8000/equilibrium", ["Content-Type" => "application/json"], body)
r = JSON3.read(resp.body)
r.results.phases[1].fraction
```

For a typed client in Java, C#, Go, Rust, TypeScript and others, feed the server's description to openapi-generator:

```
curl -o openapi.json http://127.0.0.1:8000/openapi.json
npx @openapitools/openapi-generator-cli generate -i openapi.json -g java -o calphad-client-java
```

Fortran or other codes without an HTTP library can read the CSV from `export_parabolic_csv` and port the dozen lines of formulas in `kks_parabolic.hpp`.

## 11. Validation, self-test and the 1-D demo

`selftest.py` checks relations that must hold for any database, the KKS module was checked against pycalphad's own equilibrium, and the 1-D demo runs the whole chain and reproduces the expected solidification physics.

### 11.1 `selftest.py`

At each alloy and temperature it checks:

| Check | Tolerance |
|---|---|
| Phase fractions sum to 1 | $10^{-6}$ |
| Mass balance: $\Sigma\ f_p\ x_{p,i}$ equals the alloy composition for every element | $10^{-6}$ |
| Every stable phase lies on the equilibrium tangent plane: $G_p = \Sigma\ x_{p,i}\ \mu_i$ | $10^{-3}$ J/mol |
| $\Sigma\ f_p\ G_p$ equals `GM_system` | $10^{-3}$ J/mol |
| No single phase lies below the equilibrium energy (API 2) | $10^{-3}$ J/mol |
| No driving force is positive; stable phases have zero | $10^{-3}$ and $10^{-2}$ J/mol |

With no arguments it runs 28 built-in cases: Cu–Ni at four compositions and four tem-

peratures, and Al–Cr–Ni at four compositions and three temperatures. On your own database, give the file, the solutes as JSON, the balance element and T:

```
python selftest.py alni_dupin_2001.tdb '{"AL": 0.25}' NI 1600
#   ok  alni_dupin_2001 T=  1600  {"AL": 0.25} bal NI: FCC_L12 1.000
# all 1 cases passed
```

In Windows PowerShell, quote the JSON as `'{\"AL\": 0.25}'`. A failure stops with an `AssertionError` naming the check.

## 11.2 KKS checks

| Check | Result |
|---|---|
| KKS tie lines against pycalphad equilibrium (Cu–Ni; $\gamma$/L and $\gamma'$/L in Ni–Al) | Agree to $10^{-5}$ or better |
| `split`: mass balance | $10^{-16}$ |
| `split`: equal diffusion potentials | $10^{-8}$ J/mol |
| `table` against `exact` energies | Within about $10^{-3}$ J/mol |
| `parabolic` against `exact` driving force | Identical on the tie line; 0.4% apart at 0.037 from it |
| `kks_parabolic.hpp` against the Python parabolic model | Identical to every printed digit |

## 11.3 The 1-D Ni–10Al demo

`examples/kks_1d_directional.py` solidifies Ni–10Al in a temperature gradient moving at the scan speed, the frozen-temperature setting of a melt pool:

```
python examples/kks_1d_directional.py --mode parabolic     # or exact, table
```

| Parameter | Value |
|---|---|
| Alloy c0 | Ni–10Al ($x_{\mathrm{Al}} = 0.10$) |
| Phases | `FCC_A1` ($\phi = 1$), `LIQUID` |
| Thermal gradient G, isotherm speed V | $10^7$ K/m, 0.01 m/s |
| Domain | 600 cells $\times$ 5 nm = 3 $\mu$m |
| Interface width $2\lambda$, energy $\sigma$ | 25 nm, 0.3 J/m$^2$ (illustrative) |
| $D_L$, $D_S$ | $3 \times 10^{-9}$ and $10^{-13}$ m$^2$/s |
| Molar volume | $7.0 \times 10^{-6}$ m$^3$/mol |
| Mobility M | $D_L$ / $\varepsilon^2$ (a numerical choice) |
| Simulated time | 180 $\mu$s, 107 999 steps |

Expected output:

```
Ni-10Al: solidus 1717.82 K, liquidus 1719.98 K, k(T_sol) = 0.9121, c0/k = 0.1096
mode=parabolic: 107999 steps in 15.4 s (143 us/step)
```

```
solute drift -7.0e-14
final: interface T 1717.33 K (solidus 1717.82), solid c 0.0996 (c0 0.1), interface liquid c 0.1090
    (c0/k 0.1096), interface speed 0.0099 m/s (V 0.01)
```

| **Expectation for a planar front at steady state** | **Result** |
|---|---|
| Solute conserved | Drift $-7.0 \times 10^{-14}$ |
| Front moves with the isotherms | 0.0099 m/s against 0.01 m/s |
| Interface sits at the solidus of c0 | 1717.33 K, 0.49 K below 1717.82 K: kinetic undercooling from the finite mobility |
| Solid forms at c0 | 0.0996, still approaching 0.10 |
| Liquid at the front reaches c0 / k | 0.1090, approaching 0.1096 |

`--mode exact` gives the same physics (interface at 1717.32 K) at 56 s, 521 $\mu$s per step. The script also writes `kks_1d_<mode>.png`: composition profiles, interface temperature and the solid and interface-liquid compositions over time.

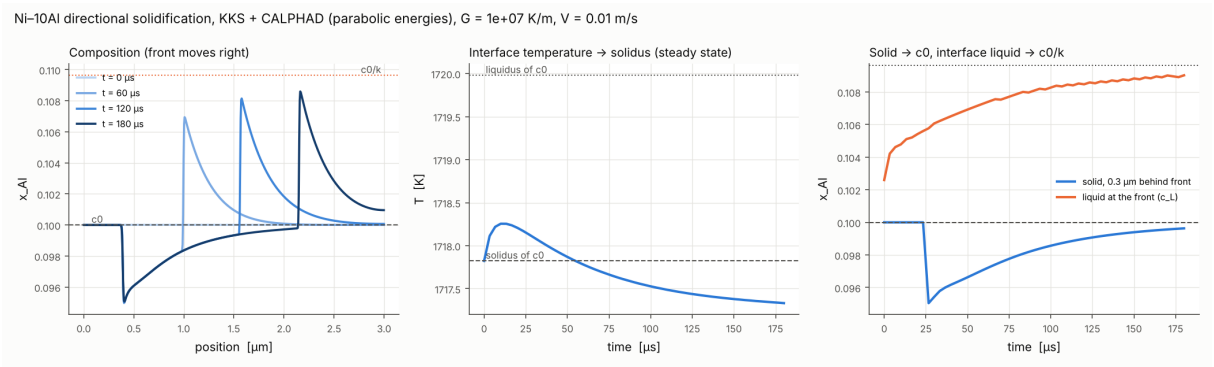



Figure 2: Ni–10Al demo output (parabolic mode): composition profiles at four times, interface temperature settling below the solidus, and solid and interface-liquid compositions approaching c0 and c0/k

# 12. Troubleshooting, performance and limitations

Most problems come from a missing `db`, a phase name that differs between databases, or a phase hidden inside its ordered partner. The table maps symptoms to fixes.

| Symptom | Likely cause | Fix |
|---|---|---|

## Troubleshooting

| Symptom | Likely cause | Fix |
|---|---|---|
| `Several databases loaded ...; say which with 'db'.` | More than one database is loaded | Add `"db"` to the request, or set `db=` on the client |
| `Unknown phase(s) [...]` | Names differ between databases (`FCC_L12` in one, `L12_FCC` in another) | `GET /databases/{db}` lists the names |
| A phase you expected is missing from the results | It is reported under its ordered partner | Read `equivalent_to`; `FCC_L12` with `equivalent_to: "FCC_A1"` is $\gamma$ |
| `stability_check.verified` is `false` | Even 10 000 samples per phase left a positive driving force | Compare candidates with API 2, look at API 3 in points mode, or restrict `phases` |
| API 1 takes several seconds | The check re-solved with denser sampling (`sampling_density` 2000 or 10000) | Expected for hard cases; `verify: false` skips it when certainty is not needed |
| `... has sublattices or vacancies mixing with the elements ... Use mode='table'.` | `mode="exact"` with an ordered phase or compound | `mode="table"` or `"parabolic"`, with `branches` for order/disorder phases |
| `mode='parabolic' needs temperatures` | No `T` given | `T=(Tmin, Tmax, step)` or a list |
| `(...) never coexist in ... K.` | The temperature range misses the two-phase field | Find the field with an API 1 sweep first |
| A tie line has `consistent: false` | The KKS energies picked a different two-phase field or branch than pycalphad | Set `c_alloy`; set `branches` |
| `kks.warnings` mentions a kink | An order/disorder phase on the default `"lowest"` branch | `branches={"FCC_L12": "ordered"}` |
| `split` info contains a `warning` | Newton did not converge, usually at c far from both phases | Warm-start with `ca0`, `cb0`; reduce the time step |
| `Database lacks atomic masses; use mole fractions.` | Weight units with a TDB that has no masses | Use mole fractions or atomic percent |
| C++: `calphad server unreachable` | Server not running, or wrong host or port | Start it; check `GET /health` in a browser |
| HTTP 422 | A required field is missing or has the wrong type | Compare the request with `/docs` |

## Performance

- Keep the server running. The first call for a new set of elements and phases compiles the models in about a second; later calls reuse them.
- In Python, import the modules directly instead of calling the server: an HTTP round trip costs about a millisecond plus JSON encoding.
- Never call the server per grid point. Fetch KKS parameters once, then evaluate locally.
- Warm-start `split` with the previous step's `ca`, `cb`: 0.22 s falls to 0.06 s on a 512 × 512 grid in exact mode. Use `parabolic` (0.03 s) for large 2-D and 3-D runs.
- Build expensive KKS models once: $\gamma'$ in `table` mode takes about 14 s. `export_json` or `export_parabolic_csv` saves the result for later runs.
- For energy surfaces over HTTP, prefer `step` $\geq 0.01$ for ternaries.

| **Operation** | **Time on the test machine** |
|---|---|
| API 1, Cu–Ni, first call | 0.5 s |
| API 1, Ni–Al, six temperatures | 1.3 s |
| API 3, Al–Cr–Ni, `step` 0.05 | 0.07 s |
| `KKSModel` exact, $\gamma$/L | 0.04 s |
| `KKSModel` parabolic, 36 temperatures | 4.5 s |
| `KKSModel` $\gamma'$ table, 13 temperatures | 14 s |
| 1-D demo, 108 000 steps, 600 cells | 15 s parabolic, 56 s exact |

## Limitations

- The KKS layer handles two phases in a binary alloy. Multicomponent or multiphase KKS is not included.
- Anti-trapping currents and the thin-interface calibration of the mobility M are left to your solver.
- `exact` mode handles substitutional phases only, and fixes P at 101 325 Pa.
- The molar volume is a constant you supply; it is not read from the database.
- The stability check catches missed phases but not every missed miscibility gap inside a phase that is already stable.
- Only substitutional and sublattice (CEF) models were tested. Ionic liquids and other models work as far as pycalphad supports them.
- The server has no authentication, and calls on one database run one at a time.
- `alcrni.tdb` is a pycalphad test database, useful for exercising ordered phases but not for alloy design.

## pycalphad pitfalls the framework works around

| **Pitfall** | **Seen in testing** | **What the framework does** |
|---|---|---|
| The minimiser stops in a local minimum when sampling misses a phase | Ni–20Al–10Cr at 1600 K: $L1_2$ + liquid instead of $L1_2$ + B2, 11 J/mol too high | Stability check with denser re-sampling (API 1) |
| A single ordered phase is returned in its metastable disordered state | Ni–25Al $\gamma'$ at 1600 K, 355 J/mol too high | Multi-start minimisation (API 2, KKS tables) |
| The local driving-force search underestimates | $\gamma'$ about 11 J/mol low | Larger of pycalphad's value and a sampled estimate |
| Disordered $\gamma$ is named after its ordered partner | `FCC_L12` for $\gamma$ | `state` and `equivalent_to` labels |

Tested with Python 3.13 and pycalphad 0.11.2.